

Module 3, Lab 2 — Textbook Problem 2.1: Outliers in the Crime Data

Goal: Screen the `usarrests.csv` dataset for values that cannot be true, and distinguish *impossible* values from *merely extreme* ones, following the count → investigate → document rhythm from Lab 1.

Repo: *(add your GitHub repo URL here before submitting, e.g. <https://github.com/roee0910/cs82a-portfolio.git>)

Step 1 — Load the data and take a first look

In [1]: `import pandas as pd`

```
df = pd.read_csv('usarrests.csv', index_col=0)
df.index.name = 'State'
df.head()
```

Out [1]:

	Murder	Assault	UrbanPop
State			
Alabama	13.2	236.0	58
Alaska	10.0	263.0	48
Arizona	8.1	294.0	80
Arkansas	8.8	190.0	50
California	9.0	276.0	91

In [2]: `df.info()`

```
<class 'pandas.DataFrame'>
Index: 50 entries, Alabama to Wyoming
Data columns (total 3 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Murder      50 non-null     float64
1   Assault     49 non-null     float64
2   UrbanPop    50 non-null     int64
dtypes: float64(2), int64(1)
memory usage: 1.6+ KB
```

`info()` already gives us one clue: **Assault has only 49 non-null values out of 50** — one state is missing a value. We'll come back to that in the missing-data check below.

Step 2 — Count: summary statistics for every numeric column

In [3]: `df.describe()`

```
Out [3]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; }
.dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; }
</style> <thead> <th></th> <th>Murder</th> <th>Assault</th> <th>UrbanPop</th>
</thead> <tbody> <th>count</th> <th>mean</th> <th>std</th> <th>min</th>
<th>25%</th> <th>50%</th> <th>75%</th> <th>max</th> </tbody> <tbody>
</tbody>
```

50.00000	49.000000	50.00000
7.78800	182.183673	74.20000
4.35551	130.877435	73.40828
0.80000	45.000000	6.00000
4.07500	109.000000	53.25000
7.25000	159.000000	66.00000
11.25000	249.000000	77.75000
17.40000	879.000000	570.00000

<p>Reading the summary table:</p>

- **Murder** (per 100,000): min 0.8, max 17.4 — a plausible range for a rate.
- **Assault** (per 100,000): min 45, max **879** — the max is more than 2.5x the 75th percentile (249) and far above the next-highest value. Worth investigating.
- **UrbanPop** (% of population that is urban): min **6**, max **570** — a percentage cannot exceed 100, so the max is a hard, structural error. The min of 6 is legal (0–100) but very low for a real state and worth a second look.

<p>This is exactly the situation the problem describes: describe() screens, it doesn't judge. The next step is to look at the actual rows behind these numbers.</p>

<h2 id="Step-3-%E2%80%94Investigate:-pull-out-each-suspicious-row">Step 3 — Investigate: pull out each suspicious row</h2>

```
In [4]: # Flag 1: UrbanPop can never exceed 100 (it's a percentage) – check for v
df[df['UrbanPop'] > 100]
```

```
Out [4]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; }
.dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; }
</style> <thead> <th></th> <th>Murder</th> <th>Assault</th> <th>UrbanPop</th>
<th>State</th> <th></th> <th></th> <th></th> </thead> <tbody> <th>iowa</th>
</tbody> <tbody></tbody>

2.2  56.0  570
```

```
In [5]: # Flag 2: look at the lowest UrbanPop values for context
df.sort_values('UrbanPop').head(3)
```

```
Out [5]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; }
.dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; }
</style> <thead> <th></th> <th>Murder</th> <th>Assault</th> <th>UrbanPop</th>
<th>State</th> <th></th> <th></th> <th></th> </thead> <tbody> <th>New York</th>
<th>Vermont</th> <th>West Virginia</th> </tbody> <tbody></tbody>
```

11.1	254.0	6
2.2	48.0	32
5.7	81.0	39

```
In [6]: # Flag 3: look at the highest Assault values for context
df.sort_values('Assault', ascending=False).head(3)
```

```
Out [6]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; }
.dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; }
</style> <thead> <th></th> <th>Murder</th> <th>Assault</th> <th>UrbanPop</th>
<th>State</th> <th></th> <th></th> <th></th> </thead> <tbody> <th>South
Carolina</th> <th>North Carolina</th> <th>Florida</th> </tbody> <tbody></tbody>
```

14.4	879.0	48
13.0	337.0	45
15.4	335.0	80

```
In [7]: # Flag 4: any missing values in the dataset?
df[df.isna().any(axis=1)]
```

```
Out [7]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; }
.dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; }
</style> <thead> <th></th> <th>Murder</th> <th>Assault</th> <th>UrbanPop</th>
<th>State</th> <th></th> <th></th> <th></th> </thead> <tbody> <th>Georgia</th>
</tbody> <tbody></tbody>
```

17.4	NaN	60
------	-----	----

<h2 id="Step-4-%E2%80%94-Judge:-impossible-vs.-extreme,-and-the-action-taken-for-each-flag">Step 4 — Judge: impossible vs. extreme, and the action taken for each flag</h2> <thead> <th>#</th> <th>State</th> <th>Column</th> <th>Value</th> <th>Verdict</th> <th>Reasoning & action</th> </thead> <tbody> </tbody> <tbody></tbody>

1	Iowa	UrbanPop	570	Impossible
---	------	----------	-----	------------

A percentage cannot exceed 100, so 570 cannot be a real value — it violates a hard constraint, not just a statistical pattern. It looks like a data-entry error (e.g., an extra digit typed, or a decimal point dropped from something like 57.0). Since the true value can't be recovered from this file, the value is set to missing (NaN) rather than guessed at, and the change is logged below.

2	New York	UrbanPop	6	Extreme, not impossible	6% is a technically valid percentage (0–100), so it doesn't break any hard rule. But New York is one of the most urbanized states in the country, so a value of 6% is implausible on domain grounds — it looks like a truncated typo (plausibly missing a leading digit, e.g. 86 → 6). Because it's still <i>possible</i> rather than <i>impossible</i> , the value is kept as-is but flagged in the log for manual verification against the original source rather than deleted or altered.
3	South Carolina	Assault	879	Extreme, not impossible	Assault is a rate per 100,000 with no fixed upper ceiling, so a high value is not a logical contradiction the way <code>UrbanPop > 100</code> is. 879 is nonetheless far outside the pattern of every other state (next highest is 337), so it is flagged as a statistical outlier. It is kept in the dataset — outliers are real data points until proven otherwise — but noted for follow-up verification against a primary source.
4	Georgia	Assault	missing (NaN)	Missing, not an outlier	Not a suspicious <i>value</i> — there's simply no value recorded. Left as <code>NaN</code> rather than filled in with a guess (e.g. mean imputation), since inventing a number would misrepresent this state's true value. Documented here so it isn't mistaken for a silently dropped row later.

<p>**Why the distinction matters:** rows 1 is a certainty — no legitimate reading of the world makes `UrbanPop = 570` possible, so correcting it (to missing, since we can't recover the true figure) is safe. Rows 2–3 are judgment calls: they are statistically unusual but not logically impossible, so the responsible move is to flag and investigate rather than silently delete or "fix" them — deleting a real extreme value would bias the dataset just as much as leaving in a bad one.</p>

<h2 id="Step-5-%E2%80%94Document:-apply-the-one-action-that's-warranted-(Iowa)-and-log-it">Step 5 — Document: apply the one action that's warranted (Iowa) and log it</h2>

```
In [8]: # Cleaning log
# -----
# 1. Iowa / UrbanPop: raw value 570 is impossible (>100%). No source is
#    available to recover the true figure, so it is set to NaN (missing)
#    rather than guessed. -- ACTION TAKEN
# 2. New York / UrbanPop = 6: valid range, but domain-implausible for a
#    highly urban state. Left unchanged; flagged for manual verification
#    against the original source. -- NO ACTION, FLAGGED ONLY
# 3. South Carolina / Assault = 879: valid range, statistically extreme.
#    Left unchanged; flagged for manual verification. -- NO ACTION, FLAGG
# 4. Georgia / Assault: missing value, left as NaN (not imputed). -- NO A
# -----
```

```
df_clean = df.copy()
df_clean['UrbanPop'] = df_clean['UrbanPop'].astype(float)
df_clean.loc['Iowa', 'UrbanPop'] = float('nan')

print("Value at Iowa/UrbanPop before:", df.loc['Iowa', 'UrbanPop'])
print("Value at Iowa/UrbanPop after: ", df_clean.loc['Iowa', 'UrbanPop'])
```

Value at Iowa/UrbanPop before: 570

Value at Iowa/UrbanPop after: nan

<h2 id="Step-6-%E2%80%94Re-check-the-summary-statistics-after-cleaning">Step 6 — Re-check the summary statistics after cleaning</h2>

In [9]: df_clean.describe()

Out [9]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; }
 .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; }
 </style> <thead> <th></th> <th>Murder</th> <th>Assault</th> <th>UrbanPop</th>
 </thead> <tbody> <th>count</th> <th>mean</th> <th>std</th> <th>min</th>
 <th>25%</th> <th>50%</th> <th>75%</th> <th>max</th> </tbody> <tbody>
 </tbody>

50.000000	49.000000	49.000000
7.78800	182.183673	64.081633
4.35551	130.877435	16.592966
0.80000	45.000000	6.000000
4.07500	109.000000	53.000000
7.25000	159.000000	66.000000
11.25000	249.000000	77.000000
17.40000	879.000000	91.000000

<p>After removing the one impossible value, UrbanPop's max drops from 570 to a plausible 91, and its count drops to 49 (one state now genuinely missing, matching Assault's missing count of 49). Assault's max (879, South Carolina) and UrbanPop's min (6, New York) are unchanged, because both were judged *extreme, not impossible* — they stay in the data, flagged for a human to verify against the original source rather than deleted.</p>

<h2 id="Summary">Summary</h2>

- **Screened** all three numeric columns with `describe()`.
- **Caught** one impossible value: Iowa's `UrbanPop = 570` (a percentage over 100).
- **Investigated** two extreme-but-possible values (New York's `UrbanPop = 6`, South Carolina's `Assault = 879`) and one missing value (Georgia's `Assault`).
- **Documented** every decision in the cleaning log above: only the impossible value was changed (to missing); the extreme values and the pre-existing missing value

were left alone and flagged for follow-up, per the investigate-before-deleting rule.